TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: HỌC SÂU

ĐỀ TÀI: HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH (BT03)

Sinh viên thực hiện: Nguyễn Phước Sang

Mã số sinhv viên: 3123411255

Lớp: DCT123C3

Giảng viên hướng dẫn: Đỗ Như Tài

BÀI GIẢI

Câu 1

Nhãn thực: A, B, C, A, B, C, A, B, C, A

Nhãn dự đoán: A, C, C, B, B, A, A, B, C, C

a) Ma trận nhầm lẫn:

Thực \ Dự đoán	A	B	C
A	2	1	1
B	0	2	1
C	1	0	2

b) Công thức: Accuracy = số dự đoán đúng / tổng số mẫu
Accuracy = (2 + 2 + 2)/10 = 0,6 = 60%

c) Công thức:
Precision = TP/(TP + FP)
Recall = TP/(TP + FN)
F1-score = 2 × Precision × Recall/(Precision + Recall)

Lớp	Precision	Recall	F1-score
A	TP/(TP+FP) = 2/(2+1) = 0,667	TP/(TP+FN) = 2/(2+2) = 0,500	2PR/(P+R) = 0,571
B	TP/(TP+FP) = 2/(2+1) = 0,667	TP/(TP+FN) = 2/(2+1) = 0,667	2PR/(P+R) = 0,667
C	TP/(TP+FP) = 2/(2+2) = 0,500	TP/(TP+FN) = 2/(2+1) = 0,667	2PR/(P+R) = 0,571

Câu 2

Nhãn thực: P, N, P, P, N, N, P, N, P, N

Nhãn dự đoán: P, N, N, P, P, N, P, N, N, P

a) Ma trận nhầm lẫn:

Thực \ Dự đoán	P	N
P	3	2
N	2	3

b) TP = 3; FP = 2; FN = 2; TN = 3

c) Công thức: Accuracy = (TP + TN)/(TP + TN + FP + FN)
Accuracy = (3 + 3)/(3 + 3 + 2 + 2) = 6/10 = 0,6 = 60%

Công thức: Precision = TP/(TP + FP)
Precision = 3/(3 + 2) = 0,6 = 60%

Công thức: Recall = TP/(TP + FN)
Recall = 3/(3 + 2) = 0,6 = 60%

Công thức: F1-score = 2 × Precision × Recall/(Precision + Recall)
F1-score = 2 × 0,6 × 0,6/(0,6 + 0,6) = 0,6 = 60%

Câu 3

Áp dụng lần lượt các luật: overcast → yes; windy = TRUE → no; sunny → no; còn lại → yes.

a) Kết quả dự đoán:

STT	Nhãn thật	Dự đoán
1	no	no
2	no	no
3	yes	yes
4	yes	yes
5	yes	yes
6	no	no
7	yes	yes
8	no	no
9	yes	no
10	yes	yes
11	yes	no
12	yes	yes
13	yes	yes
14	no	no

b) Chọn lớp dương là play = yes:

Thực \ Dự đoán	yes	no
yes	7	2
no	0	5

TP = 7; FP = 0; FN = 2; TN = 5

c) Các chỉ số:

Công thức: Accuracy = (TP + TN)/(TP + TN + FP + FN)
Accuracy = (7 + 5)/14 = 12/14 ≈ 0,857 = 85,7%

Công thức: Precision = TP/(TP + FP)
Precision = 7/(7 + 0) = 1 = 100%

Công thức: Recall (Sensitivity) = TP/(TP + FN)
Recall = 7/(7 + 2) = 7/9 ≈ 0,778 = 77,8%

Công thức: Specificity = TN/(TN + FP)
Specificity = 5/(5 + 0) = 1 = 100%

Công thức: F1-score = 2 × Precision × Recall/(Precision + Recall)
F1-score = 2 × 1 × 0,778/(1 + 0,778) ≈ 0,875 = 87,5%

1. Trình bày các phương pháp học được giới thiệu.

Học có giám sát: học từ dữ liệu có nhãn, ví dụ dự đoán giá nhà hoặc phân loại thư rác. Học không giám sát:

tìm cấu trúc trong dữ liệu chưa có nhãn, ví dụ phân cụm khách hàng. Học bán giám sát: kết hợp ít dữ liệu

có nhãn với nhiều dữ liệu chưa có nhãn. Học tăng cường: tác nhân thử hành động, nhận phần thưởng và

học chính sách để tối đa hóa phần thưởng.

2. Định nghĩa giả thuyết và không gian giả thuyết.

Giả thuyết h là một hàm/mô hình ứng viên ánh xạ đầu vào x thành dự đoán h(x). Không gian giả thuyết H

là tập tất cả các mô hình ứng viên mà thuật toán có thể chọn. Ví dụ, nếu chỉ xét đường thẳng y = ax + b thì

H gồm mọi đường thẳng ứng với các giá trị a, b được phép.

3. Phân biệt biến dự báo và biến mục tiêu.

Biến dự báo (X) là thông tin đầu vào dùng để dự đoán. Biến mục tiêu (y) là kết quả cần dự đoán. Ví dụ, diện

tích và số phòng là X; giá nhà là y.

4. Hàm mục tiêu (target function) là gì?

Hàm mục tiêu f là quy luật thực sự liên hệ đầu vào với kết quả mong muốn: y = f(x) (có thể kèm nhiễu). Ta

thường không biết chính xác f nên dùng dữ liệu để học một giả thuyết h xấp xỉ f. Không nhầm hàm mục

tiêu này với hàm mất mát dùng để tối ưu mô hình.

5. Định nghĩa học khái niệm (concept learning).

Học khái niệm là bài toán rút ra một quy tắc phân loại từ các ví dụ dương và âm để xác định một đối tượng

có thuộc khái niệm cần học hay không. Ví dụ, học quy tắc nhận biết một email là thư rác.

6. Khái niệm được hiểu thế nào trong học khái niệm?

Một khái niệm là một tập con các đối tượng trong miền dữ liệu, hoặc tương đương là hàm Boolean c(x)

nhận 1 nếu x thuộc khái niệm và 0 nếu không thuộc. Ví dụ, “ngày phù hợp để chơi tennis” là một khái niệm

trên tập các ngày với thuộc tính thời tiết.

7. Nêu ba thành phần cần thiết của một bài toán học khái niệm.

Cần có: (1) tập các trường hợp/đối tượng X được mô tả bằng thuộc tính; (2) khái niệm mục tiêu c cần học;

(3) tập ví dụ huấn luyện D gồm các trường hợp x và nhãn c(x). Thuật toán sẽ chọn giả thuyết h trong không

gian H để khớp dữ liệu và dự đoán trường hợp mới.

8. Trình bày cách biểu diễn một giả thuyết.

Có thể biểu diễn h dưới dạng phép hội của các ràng buộc trên thuộc tính. Mỗi thuộc tính nhận một giá trị cụ

thể, dấu ? (chấp nhận mọi giá trị), hoặc ký hiệu (không chấp nhận giá trị nào). Ví dụ <Trời=nắng, Gió=?,

Nhiệt độ=ấm> mô tả các ngày nắng, ấm, bất kể gió.

9. Định nghĩa phép quy nạp và thiên lệch quy nạp.

Quy nạp là suy ra quy luật tổng quát từ số lượng ví dụ hữu hạn để dự đoán trường hợp chưa gặp. Thiên lệch

quy nạp là các giả định giúp thuật toán chọn một số quy luật thay vì những quy luật khác, ví dụ ưu tiên mô

hình đơn giản. Không có giả định này, dữ liệu hữu hạn không xác định được duy nhất cách dự đoán mọi

trường hợp mới.

10. Nêu ba loại sai số dự đoán được trình bày.

Thường tách sai số dự đoán thành: sai lệch (bias) do giả định mô hình quá đơn giản; phương sai (variance)

do mô hình nhạy với bộ dữ liệu huấn luyện; và nhiễu không thể loại bỏ (irreducible noise) do yếu tố ngẫu

nhiên hoặc thông tin chưa quan sát. Với mất mát bình phương: $E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$.

11. Giải thích sự đánh đổi giữa bias và variance.

Mô hình quá đơn giản thường có bias cao, variance thấp; mô hình quá phức tạp thường có bias thấp,

variance cao. Mục tiêu là chọn độ phức tạp giúp sai số trên dữ liệu mới thấp nhất, thường bằng đánh giá

trên tập validation hoặc cross-validation.

12. Phân biệt overfitting và underfitting.

Underfitting: mô hình học chưa đủ quy luật, sai số cao trên cả dữ liệu huấn luyện và kiểm tra. Overfitting:

mô hình học cả nhiễu của tập huấn luyện, sai số huấn luyện thấp nhưng sai số kiểm tra cao. Có thể giảm

overfitting bằng thêm dữ liệu, regularization, giảm độ phức tạp hoặc dừng sớm.

13. Phân biệt tham số mô hình và siêu tham số.

Tham số mô hình được học từ dữ liệu trong quá trình huấn luyện, ví dụ hệ số w, b của hồi quy tuyến tính

hoặc trọng số mạng nơ-ron. Siêu tham số do người xây dựng đặt/chọn trước quá trình học, ví dụ learning

rate, độ sâu cây, k của k-NN; có thể chọn qua validation/cross-validation.

HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

14. Trình bày các phương pháp lấy mẫu lại (re-sampling).

Hold-out: chia dữ liệu thành tập huấn luyện và tập kiểm tra (thường thêm validation). K-fold

cross-validation: chia thành K phần; lần lượt dùng 1 phần để đánh giá và K - 1 phần để huấn luyện, rồi lấy

trung bình K kết quả. LOOCV là trường hợp K bằng số mẫu. Repeated K-fold lặp lại phép chia K-fold.

Bootstrap lấy mẫu có hoàn lại nhiều lần, huấn luyện trên mẫu được rút và đánh giá trên các mẫu ngoài túi

(out-of-bag) hoặc dùng để ước lượng độ bất định. Khi chọn siêu tham số, giữ tập test cuối cùng riêng để

tránh đánh giá lạc quan.

15. Trình bày khái quát các learning frameworks được giới thiệu.

Một số framework thường dùng: scikit-learn cung cấp thuật toán máy học truyền thống cùng công cụ tiền

xử lý, Pipeline và đánh giá; TensorFlow/Keras hỗ trợ xây dựng, huấn luyện và triển khai mạng nơ-ron, trong

đó Keras có API mức cao; PyTorch hỗ trợ tính toán tensor, tự động đạo hàm và xây dựng mạng nơ-ron linh

hoạt. Chọn công cụ theo loại mô hình, dữ liệu và yêu cầu triển kha

CÂU 3.

Thứ tự thuộc tính: Fever, Cough, Throat Pain, Body Pain

Mẫu dương: I1, I2, I3, I5. Mẫu âm: I4, I6, I7.

a) Không gian đầu vào, biến mục tiêu và biểu diễn giả thuyết

Không gian đầu vào gồm 4 thuộc tính Fever, Cough, Throat Pain, Body Pain; mỗi thuộc tính nhận Y hoặc N.

Biến mục tiêu: Covid-19 {Positive, Negative}.

Giả thuyết có dạng h = h1, h2, h3, h4 ; dấu ? nghĩa là chấp nhận cả Y và N.

b) Tìm giả thuyết bằng Find-S

Find-S chỉ xét các mẫu dương.

Bước	Giả thuyết S
S0	, , ,
I1 (+)	N, Y, Y, Y
I2 (+)	?, Y, Y, Y
I3 (+)	?, ?, Y, Y
I5 (+)	?, ?, Y, ?

Kết quả Find-S: S = ?, ?, Y, ? .

c) Generalization/Specialization với cả mẫu dương và mẫu âm

Khởi tạo: S0 = , , , ; G0 = { ?, ?, ?, ? }.

Mẫu	S	G
I1 (+)	N, Y, Y, Y	{ ?, ?, ?, ? }
I2 (+)	?, Y, Y, Y	{ ?, ?, ?, ? }
I3 (+)	?, ?, Y, Y	{ ?, ?, ?, ? }
I4 (-)	không đổi	{ ?, ?, Y, ? ; ?, ?, ?, Y }
I5 (+)	?, ?, Y, ?	{ ?, ?, Y, ? }
I6 (-)	không đổi	không đổi
I7 (-)	không đổi	không đổi

Sau toàn bộ dữ liệu, giả thuyết nhất quán còn lại là ?, ?, Y, ? .

d) Version space

S = G = { ?, ?, Y, ? }. Vì hai biên trùng nhau nên version space chỉ có một giả thuyết.

e) Phân loại mẫu mới

Dùng h = ?, ?, Y, ? : nếu Throat Pain = Y thì dự đoán Positive; nếu Throat Pain = N thì dự đoán Negative.

CÂU 4.

a) Training error và test error

Training error là sai số trên tập huấn luyện. Test error là sai số trên dữ liệu chưa dùng để huấn luyện, phản ánh khả năng tổng quát hóa. Training error thấp chưa chắc test error thấp vì có thể xảy ra overfitting.

b) Sample error và true error

Sample error là sai số đo trên một tập mẫu hữu hạn: $E_{sample}(h)$ = số mẫu dự đoán sai / tổng số mẫu.

True error là xác suất dự đoán sai trên phân phối dữ liệu thực: $E_{true}(h) = P(h(x) \neq y)$.

c) Holdout, K-fold, Stratified K-fold và LOOCV

Holdout: chia dữ liệu thành train/test một lần.

K-fold: chia dữ liệu thành K phần; mỗi lần dùng 1 phần để kiểm tra và K-1 phần để huấn luyện, sau đó lấy kết quả trung bình.

Stratified K-fold: giống K-fold nhưng giữ gần đúng tỉ lệ lớp trong mỗi fold.

LOOCV: với N mẫu, mỗi lần dùng N-1 mẫu để train và 1 mẫu để test; lặp N lần.

d) Vì sao dùng cross-validation?

Cross-validation giúp đánh giá mô hình ổn định hơn, hỗ trợ lựa chọn mô hình và điều chỉnh siêu tham số, đồng thời giảm phụ thuộc vào một lần chia dữ liệu.

e) PAC learning

PAC (Probably Approximately Correct) nghĩa là với xác suất ít nhất $1-\delta$, thuật toán tìm được giả thuyết có sai số không vượt quá ε khi số mẫu đủ lớn.

f) Sample complexity và computational complexity

Sample complexity là số lượng mẫu cần để học được mô hình đạt độ chính xác mong muốn.

Computational complexity là lượng tài nguyên tính toán cần dùng, thường xét thời gian và bộ nhớ.

g) Hoeffding's inequality

Một dạng thường dùng: $P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2e^{(-2N\varepsilon^2)}$.

Ý nghĩa: khi số mẫu N tăng, xác suất để sai số trong mẫu khác xa sai số ngoài mẫu giảm; vì vậy E_{in} thường tiến gần E_{out} hơn.

h) VC dimension

VC dimension là số điểm lớn nhất mà một lớp giả thuyết có thể phân loại theo mọi cách có thể. VC dimension càng lớn thì mô hình càng linh hoạt nhưng thường cần nhiều dữ liệu hơn để tổng quát hóa tốt.

CÂU 5.

Thứ tự thuộc tính: Horns, Tail, Tusks, Paws, Fur, Color, Hooves, Size

Mẫu dương: 1, 2, 4, 6. Mẫu âm: 3, 5.

a) Thuộc tính đầu vào, thuộc tính đích và biểu diễn giả thuyết

Thuộc tính đầu vào: Horns, Tail, Tusks, Paws, Fur, Color, Hooves, Size. S.No. chỉ là số thứ tự.

Thuộc tính đích: Elephant {Yes, No}.

Giả thuyết có dạng h = h1, h2, ..., h8 ; dấu ? nghĩa là chấp nhận mọi giá trị.

b) Giả thuyết tổng quát nhất và cụ thể nhất

Tổng quát nhất: ?, ?, ?, ?, ?, ?, ?, ?

Cụ thể nhất: , , , , , , ,

c) Áp dụng Find-S

Find-S chỉ cập nhật khi gặp mẫu dương; mẫu âm được bỏ qua.

Bước	Giả thuyết S
S0	, , , , , , ,
Mẫu 1 (+)	No, Short, Yes, No, No, Black, No, Big
Mẫu 2 (+)	No, Short, ?, No, No, ?, No, ?
Mẫu 3 (-)	Bỏ qua
Mẫu 4 (+)	Không đổi
Mẫu 5 (-)	Bỏ qua
Mẫu 6 (+)	No, Short, ?, ?, ?, ?, No, ?

d) Giả thuyết cuối cùng của Find-S

h = No, Short, ?, ?, ?, ?, No, ? .

Nghĩa là các mẫu dương có điểm chung chắc chắn: Horns = No, Tail = Short và Hooves = No.

e) Candidate Elimination

Khởi tạo: S0 = , , , , , , , ; G0 = { ?, ?, ?, ?, ?, ?, ?, ? }.

Mẫu dương làm S tổng quát hơn; mẫu âm làm G cụ thể hơn nhưng G vẫn phải bao phủ S.

Mẫu	S	G
1 (+)	No, Short, Yes, No, No, Black, No, Big	{ ?, ?, ?, ?, ?, ?, ?, ? }
2 (+)	No, Short, ?, No, No, ?, No, ?	{ ?, ?, ?, ?, ?, ?, ?, ? }
3 (-)	không đổi	{ No, ?, ?, ?, ?, ?, ?, ? ; ?, ?, ?, ?, ?, ?, No, ? }
4 (+)	không đổi	không đổi
5 (-)	không đổi	chuyên biệt hóa G để loại mẫu 5
6 (+)	No, Short, ?, ?, ?, ?, No, ?	{ No, Short, ?, ?, ?, ?, ?, ? ; ?, Short, ?, ?, ?, ?, No, ? }

Ở mẫu 3 âm, Horns và Hooves khác với S: S có No còn mẫu âm có Yes. Vì vậy G được chuyên biệt tối thiểu bằng Horns = No hoặc Hooves = No để loại mẫu âm.

f) Version space

S = No, Short, ?, ?, ?, ?, No, ?

G = { No, Short, ?, ?, ?, ?, ?, ? ; ?, Short, ?, ?, ?, ?, No, ? }

VS = { No, Short, ?, ?, ?, ?, No, ? ; No, Short, ?, ?, ?, ?, ?, ? ; ?, Short, ?, ?, ?, ?, No, ? }.